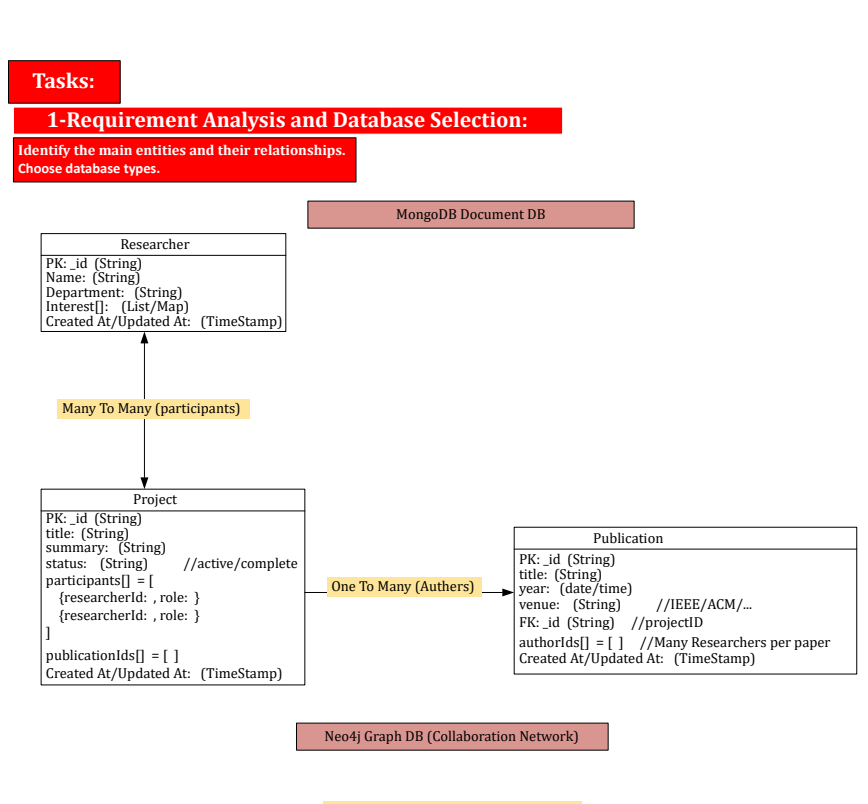
RESEARCH COLLABORATION SYSTEM (Multi NoSQL)

Implementation Project:
- Created by Student: Mohammad Alkhdour / 246008
- For NoSQL Course
- Supervised By: Dr. Hani Salah
- Year: 2026-01

Tasks:

1-Requirement Analysis and Database Selection:

Identify the main entities and their relationships.
Choose database types.



MongoDB Document DB

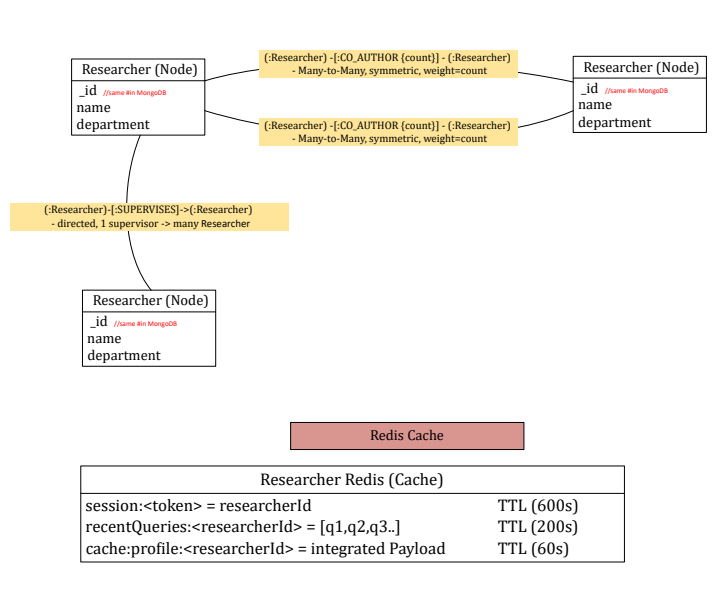
Researcher
PK: _id (String)
Name: (String)
Department: (String)
Interest[]: (List/Map)
Created At/Updated At: (TimeStamp)

Many To Many (participants)

Project
PK: _id (String)
title: (String)
summary: (String)
status: (String) //active/complete
participants[] = [
{researcherId: , role: }
{researcherId: , role: }
]
publicationIds[] = [ ]
Created At/Updated At: (TimeStamp)

One To Many (Authers)

Publication
PK: _id (String)
title: (String)
year: (date/time)
venue: (String) //IEEE/ACM/...
FK: _id (String) //projectID
authorIds[] = [ ] //Many Researchers per paper
Created At/Updated At: (TimeStamp)



Neo4j Graph DB (Collaboration Network)

Researcher (Node)
_id //same #in MongoDB
name
department

(:Researcher) -[:CO_AUTHOR {count}] - (:Researcher)
- Many-to-Many, symmetric, weight=count

(:Researcher)-[:SUPERVISES]->(:Researcher)
- directed, 1 supervisor -> many Researcher

Redis Cache

Researcher Redis (Cache)	
session:<token> = researcherId	TTL (600s)
recentQueries:<researcherId> = [q1,q2,q3..]	TTL (200s)
cache:profile:<researcherId> = integrated Payload	TTL (60s)

Justify your choices briefly.

Brief Justification:

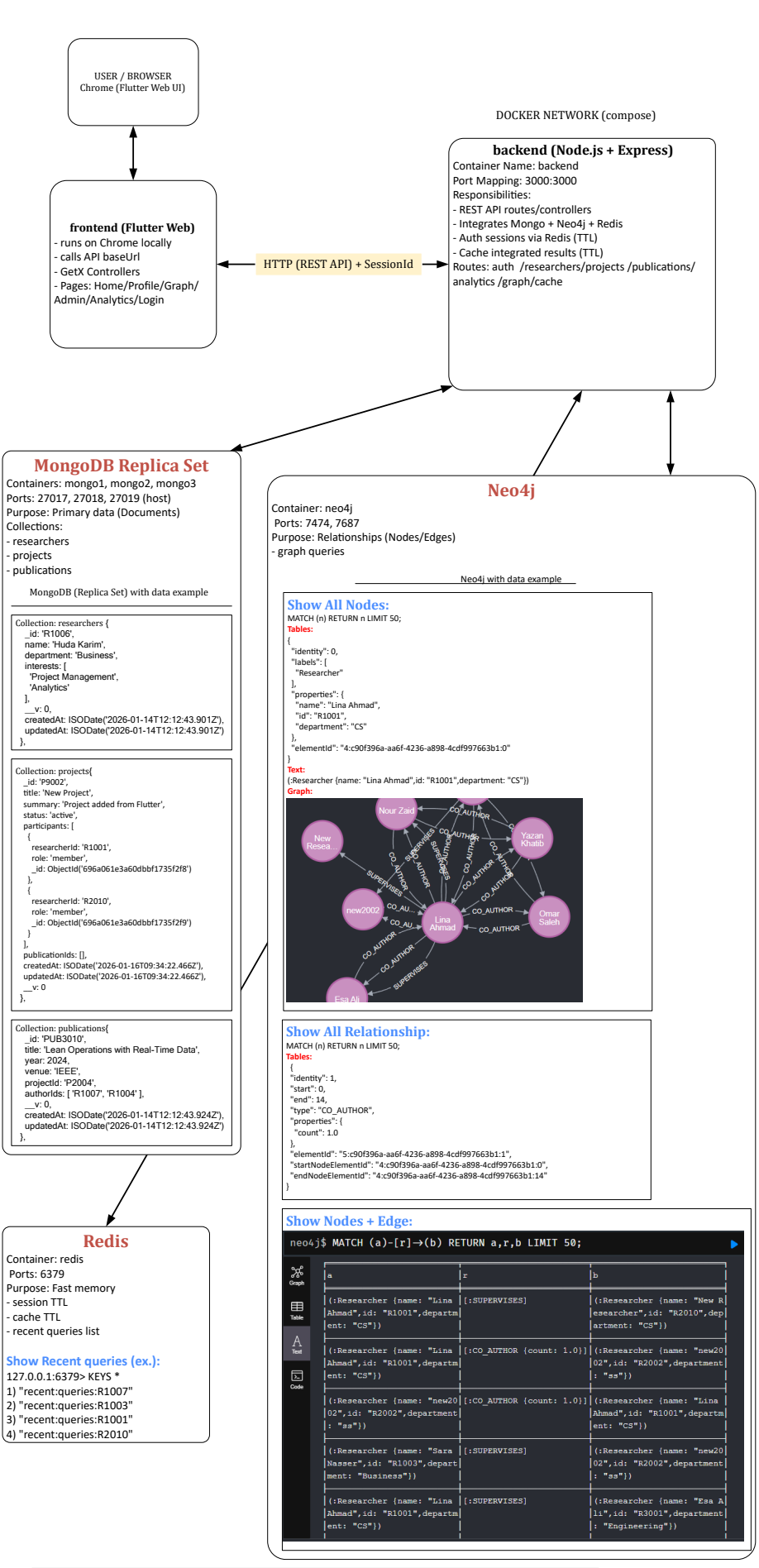
MongoDB:
Suitable for storing core entity data (Researcher / Project / Publication) because it supports flexible document-based structures. Queries such as "researcher's projects" or "researcher's publications" are straightforward using simple filters.

Neo4j:
It reduces the complexity of join/traversal operations compared to modeling the network inside MongoDB.

Redis:
Managing sessions, login info, TTL, and recent queries with very high speed.

2.System Architecture

- Design a simple architecture diagram showing how the databases interact.
- Specify which data is stored where.



USER / BROWSER
Chrome (Flutter Web UI)

frontend (Flutter Web)
- runs on Chrome locally
- calls API baseUrl
- GetX Controllers
- Pages: Home/Profile/Graph/Admin/Analytics/Login

HTTP (REST API) + SessionId

DOCKER NETWORK (compose)

backend (Node.js + Express)
Container Name: backend
Port Mapping: 3000:3000
Responsibilities:
- REST API routes/controllers
- Integrates Mongo + Neo4j + Redis
- Auth sessions via Redis (TTL)
- Cache integrated results (TTL)
Routes: auth /researchers/projects /publications/analytics /graph/cache

MongoDB Replica Set
Containers: mongo1, mongo2, mongo3
Ports: 27017, 27018, 27019 (host)
Purpose: Primary data (Documents)
Collections:
- researchers
- projects
- publications

MongoDB (Replica Set) with data example

```
Collection: researchers {
  _id: 'R1006',
  name: 'Huda Karim',
  department: 'Business',
  interests: [
    'Project Management',
    'Analytics'
  ],
  __v: 0,
  createdAt: ISODate('2026-01-14T12:12:43.901Z'),
  updatedAt: ISODate('2026-01-14T12:12:43.901Z')
},
```

```
Collection: projects{
  _id: 'P9002',
  title: 'New Project',
  summary: 'Project added from Flutter',
  status: 'active',
  participants: [
    {
      researcherId: 'R1001',
      role: 'member',
      _id: ObjectId('696a061e3a60dbbf1735f2f8')
    },
    {
      researcherId: 'R2010',
      role: 'member',
      _id: ObjectId('696a061e3a60dbbf1735f2f9')
    }
  ],
  publicationIds: [],
  createdAt: ISODate('2026-01-16T09:34:22.466Z'),
  updatedAt: ISODate('2026-01-16T09:34:22.466Z'),
  __v: 0
},
```

```
Collection: publications{
  _id: 'PUB3010',
  title: 'Lean Operations with Real-Time Data',
  year: 2024,
  venue: 'IEEE',
  projectId: 'P2004',
  authorIds: [ 'R1007', 'R1004' ],
  __v: 0,
  createdAt: ISODate('2026-01-14T12:12:43.924Z'),
  updatedAt: ISODate('2026-01-14T12:12:43.924Z')
},
```

Neo4j
Container: neo4j
Ports: 7474, 7687
Purpose: Relationships (Nodes/Edges)
- graph queries

Neo4j with data example

Show All Nodes:
MATCH (n) RETURN n LIMIT 50;
Tables:

```
{
  "identity": 0,
  "labels": [
    "Researcher"
  ],
  "properties": {
    "name": "Lina Ahmad",
    "id": "R1001",
    "department": "CS"
  },
  "elementId": "4:c90f396a-aa6f-4236-a898-4cdf997663b1:0"
}
```

Text:
(:Researcher {name: "Lina Ahmad",id: "R1001",department: "CS"})
Graph:

Show All Relationship:
MATCH (n) RETURN n LIMIT 50;
Tables:

```
{
  "identity": 1,
  "start": 0,
  "end": 14,
  "type": "CO_AUTHOR",
  "properties": {
    "count": 1.0
  },
  "elementId": "5:c90f396a-aa6f-4236-a898-4cdf997663b1:1",
  "startNodeElementId": "4:c90f396a-aa6f-4236-a898-4cdf997663b1:0",
  "endNodeElementId": "4:c90f396a-aa6f-4236-a898-4cdf997663b1:14"
}
```

Show Nodes + Edge:

```
neo4j$ MATCH (a)-[r]->(b) RETURN a,r,b LIMIT 50;
```

Redis
Container: redis
Ports: 6379
Purpose: Fast memory
- session TTL
- cache TTL
- recent queries list

Show Recent queries (ex.):

```
127.0.0.1:6379> KEYS *
1) "recent:queries:R1007"
2) "recent:queries:R1003"
3) "recent:queries:R1001"
4) "recent:queries:R2010"
```

3.Data Modeling and Implementation

- Create data models for each selected database.
- Load a small dataset (manually or through scripts).
- Implement: Basic CRUD operations. One or two meaningful queries per database type.

MongoDB insertion & Display Results

1) Researchers insertion New

```
db.researchers.insertMany([
  {
    _id: "R1001",
    name: "Ahmed Ali",
    department: "Computer Science",
    interests: ["AI", "Databases", "Data Mining"],
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    _id: "R2010",
    name: "Sara Hassan",
    department: "Information Systems",
    interests: ["NoSQL", "Graphs", "Web"],
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    _id: "R3050",
    name: "Omar Nabil",
    department: "Computer Engineering",
    interests: ["Networks", "Security"],
    createdAt: new Date(),
    updatedAt: new Date()
  }
]);
```

Researchers Display Results

```
db.researchers.find(
  { _id: { $in: ["R1001", "R2010", "R3050"] } }
).pretty()
[
  {
    _id: 'R1001',
    name: 'Ahmed Ali',
    department: 'Computer Science',
    interests: [ 'AI', 'Databases', 'Data Mining' ],
    __v: 0,
    createdAt: ISODate('2026-01-14T12:12:43.900Z'),
    updatedAt: ISODate('2026-01-18T17:06:43.243Z')
  },
  {
    _id: 'R2010',
    name: 'Sara Hassan',
    department: 'Information Systems',
    interests: [ 'NoSQL', 'Graphs', 'Web' ],
    createdAt: ISODate('2026-01-16T13:24:16.473Z'),
    updatedAt: ISODate('2026-01-18T17:06:43.257Z'),
    __v: 0
  },
  {
    _id: 'R3050',
    createdAt: ISODate('2026-01-18T17:06:43.263Z'),
    department: 'Computer Engineering',
    interests: [ 'Networks', 'Security' ],
    name: 'Omar Nabil',
    updatedAt: ISODate('2026-01-18T17:06:43.263Z')
  }
]
```

2) Projects insertion New

```
db.projects.insertMany([
  {
    _id: "P9002",
    title: "New Project",
    summary: "Project added from Flutter",
    status: "active",
    participants: [
      { researcherId: "R1001", role: "member" },
      { researcherId: "R2010", role: "member" }
    ],
    publicationIds: [],
    createdAt: new Date(),
    updatedAt: new Date(),
    __v: 0
  },
  {
    _id: "P9003",
    title: "Graph Collaboration Study",
    summary: "Study collaboration patterns using graph analytics",
    status: "active",
    participants: [
      { researcherId: "R2010", role: "lead" },
      { researcherId: "R3050", role: "member" }
    ],
    publicationIds: [],
    createdAt: new Date(),
    updatedAt: new Date(),
    __v: 0
  }
]);
```

Projects Display Results

```
[
  {
    _id: 'P9002',
    title: 'New Project',
    summary: 'Project added from Flutter',
    status: 'active',
    participants: [
      { researcherId: 'R1001', role: 'member' },
      { researcherId: 'R2010', role: 'member' }
    ],
    publicationIds: [],
    createdAt: ISODate('2026-01-16T09:34:22.466Z'),
    updatedAt: ISODate('2026-01-18T17:13:09.230Z'),
    __v: 0
  },
  {
    _id: 'P9003',
    title: 'Graph Collaboration Study',
    summary: 'Study collaboration patterns using graph analytics',
    status: 'active',
    participants: [
      { researcherId: 'R2010', role: 'lead' },
      { researcherId: 'R3050', role: 'member' }
    ],
    publicationIds: [],
    createdAt: ISODate('2026-01-16T13:30:02.477Z'),
    updatedAt: ISODate('2026-01-18T17:13:09.235Z'),
    __v: 0
  }
]
```

3) Publications insertion New

```
db.publications.insertMany([
  {
    _id: "PUB7001",
    title: "A Lightweight NoSQL Design for Research Collaboration Systems",
    year: 2025,
    projectId: "P9002",
    authors: ["R1001", "R2010"],
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    _id: "PUB7002",
    title: "Graph-Based Analytics for Academic Collaboration Networks",
    year: 2026,
    projectId: "P9003",
    authors: ["R2010", "R3050"],
    createdAt: new Date(),
    updatedAt: new Date()
  }
]);

// 4) Update projects with publicationIds (linking)
db.projects.updateOne(
  { _id: "P9002" },
  { $set: { publicationIds: ["PUB7001"], updatedAt: new Date() } }
);

db.projects.updateOne(
  { _id: "P9003" },
  { $set: { publicationIds: ["PUB7002"], updatedAt: new Date() } }
);
```

Publications Display Results

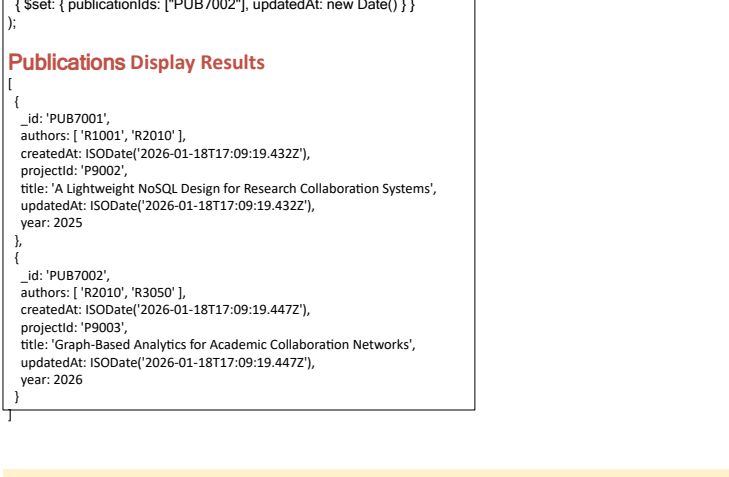
```
[
  {
    _id: 'PUB7001',
    authors: [ 'R1001', 'R2010' ],
    createdAt: ISODate('2026-01-18T17:09:19.432Z'),
    projectId: 'P9002',
    title: 'A Lightweight NoSQL Design for Research Collaboration Systems',
    updatedAt: ISODate('2026-01-18T17:09:19.432Z'),
    year: 2025
  },
  {
    _id: 'PUB7002',
    authors: [ 'R2010', 'R3050' ],
    createdAt: ISODate('2026-01-18T17:09:19.447Z'),
    projectId: 'P9003',
    title: 'Graph-Based Analytics for Academic Collaboration Networks',
    updatedAt: ISODate('2026-01-18T17:09:19.447Z'),
    year: 2026
  }
]
```

Neo4j (Insertion Data)

Add researchers Nodes

```
MERGE (r:Researcher {researcherId:'R1001'})
SET r.name='Ahmed Ali', r.department='Computer Science';

MERGE (r:Researcher {researcherId:'R2010'})
SET r.name='Sara Hassan', r.department='Information Systems';

MERGE (r:Researcher {researcherId:'R3050'})
SET r.name='Omar Nabil', r.department='Computer Engineering';
```

Display Results



```
MATCH (r:Researcher)
WHERE r.researcherId IN ['R1001','R2010','R3050']
RETURN r;
```

```
r
(:Researcher {researcherId: "R1001",name: "Ahmed Ali",department: "Computer Science"})
(:Researcher {researcherId: "R2010",name: "Sara Hassan",department: "Information Systems"})
(:Researcher {researcherId: "R3050",name: "Omar Nabil",department: "Computer Engineering"})
```

Add Collaboration Network (edges)

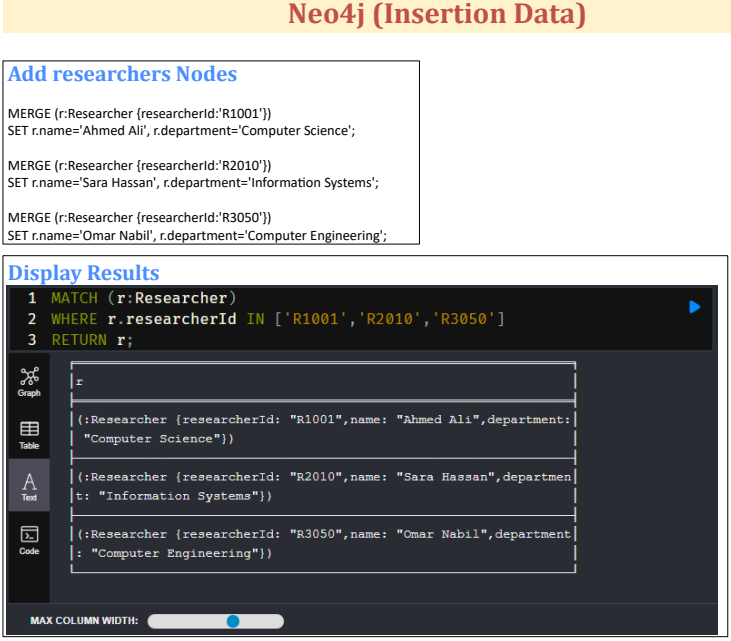
co-authorship

```
MATCH (a:Researcher {researcherId:'R1001'}), (b:Researcher {researcherId:'R2010'})
MERGE (a)-[r:CO_AUTHOR]-(b)
ON CREATE SET r.count = 1
ON MATCH SET r.count = r.count + 1;
```

Supervision

```
MATCH (s:Researcher {researcherId:'R2010'}), (t:Researcher {researcherId:'R1001'})
MERGE (s)-[:SUPERVISES]->(t);
```

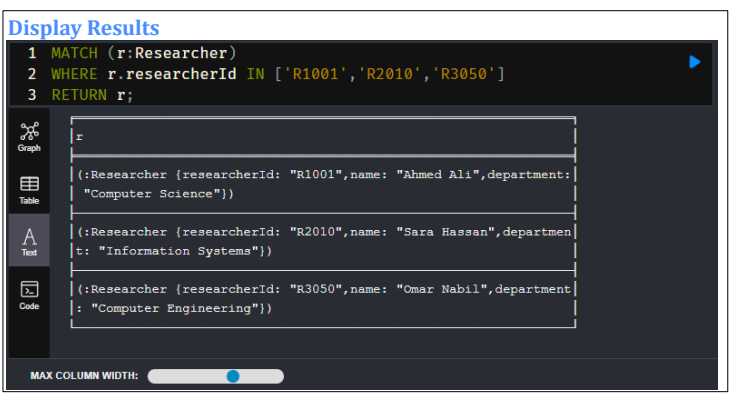
Display Results - CO_AUTHOR



```
MATCH (a:Researcher {researcherId:'R1001'})-[r:CO_AUTHOR]-
(b:Researcher {researcherId:'R2010'})
RETURN a,r,b;
```

```
a | r | b
(:Researcher {researcherId: "R1001",name: "Ahmed Ali",department: "Computer Science"}) | [:CO_AUTHOR {count: 1}] | (:Researcher {researcherId: "R2010",name: "Sara Hassan",department: "Information Systems"})
```

Display Results - SUPERVISES

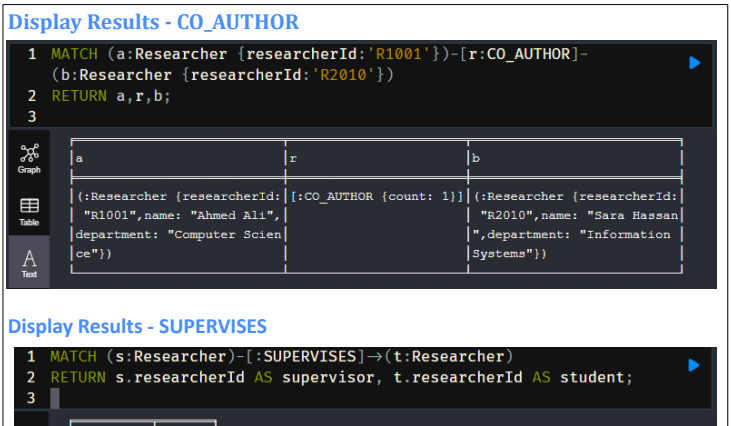


```
MATCH (s:Researcher)-[:SUPERVISES]->(t:Researcher)
RETURN s.researcherId AS supervisor, t.researcherId AS student;
```

```
supervisor | student
null       | null
null       | null
null       | null
"R2010"    | "R1001"
null       | null
null       | null
```

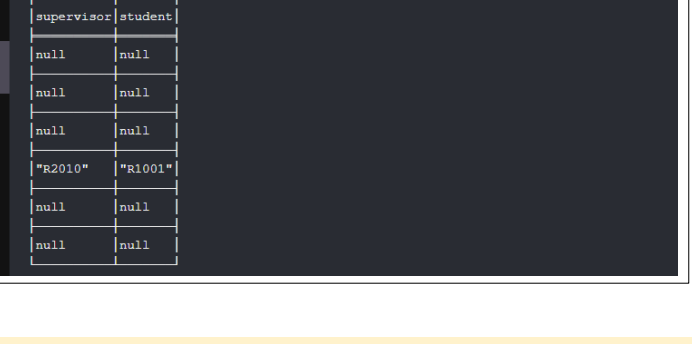
Redis (Insertion Data)

Create Sessions:
SET session:SID001 R1001 EX 3600
SET session:SID002 R2010 EX 1800
Check Session:
GET session:SID001
TTL session:SID001



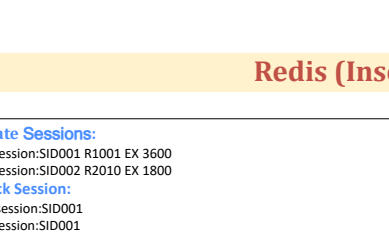
```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> docker exec -it redis redis-cli
>>
127.0.0.1:6379> FLUSHDB
(error) ERR unknown command 'FLUSHDB', with args beginning with:
127.0.0.1:6379> SET session:SID001 R1001 EX 3600
OK
127.0.0.1:6379> SET session:SID002 R2010 EX 1800
OK
127.0.0.1:6379> GET session:SID001
"R1001"
127.0.0.1:6379> TTL session:SID001
(integer) 3590
127.0.0.1:6379>
```

Create Cache:
SET cache:researcher:R1001:profile "{\"researcherId\":\"R1001\", \"name\":\"Ahmed Ali\", \"department\":\"Computer Science\" }" EX 120
SET cache:researcher:R2010:profile "{\"researcherId\":\"R2010\", \"name\":\"Sara Hassan\", \"department\":\"Information Systems\" }" EX 120
Check Cache:
GET cache:researcher:R1001:profile
TTL cache:researcher:R1001:profile

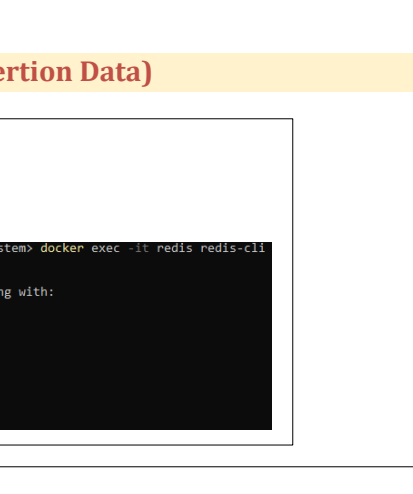


```
127.0.0.1:6379> GET cache:researcher:R1001:profile
"{ \"researcherId\":\"R1001\", \"name\":\"Ahmed Ali\", \"department\":\"Computer Science\" }"
127.0.0.1:6379> TTL cache:researcher:R1001:profile
(integer) 113
127.0.0.1:6379>
```

Recent Queries:
LPUSH recentQueries "GET researcher profile R1001"
LPUSH recentQueries "GET collaborators of R2010"
LPUSH recentQueries "GET projects of R1001"
Check Recent Queries:
LRANGE recentQueries 0 -1



```
127.0.0.1:6379> LRANGE recentQueries 0 -1
1) "GET projects of R1001"
2) "GET collaborators of R2010"
3) "GET researcher profile R1001"
4) "GET projects of R1001"
5) "GET collaborators of R2010"
6) "GET researcher profile R1001"
127.0.0.1:6379>
```

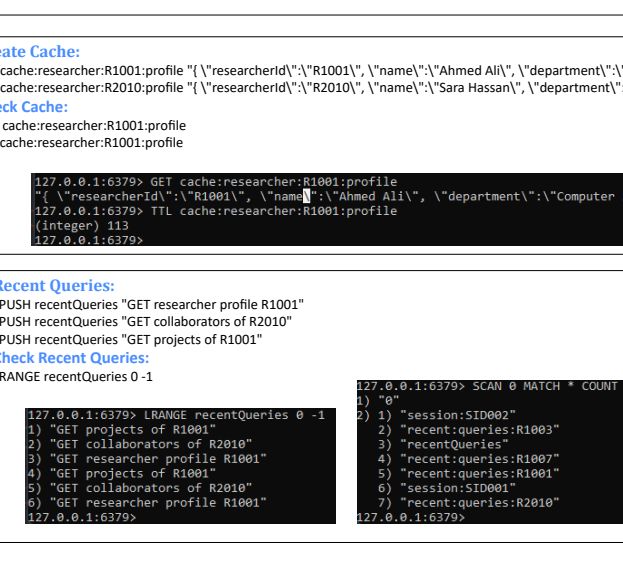


```
127.0.0.1:6379> SCAN 0 MATCH * COUNT 100
1) "0"
2) 1) "session:SID002"
   2) "recent:queries:R1003"
   3) "recentQueries"
   4) "recent:queries:R1007"
   5) "recent:queries:R1001"
   6) "session:SID001"
   7) "recent:queries:R2010"
127.0.0.1:6379>
```

- Implement: Basic CRUD operations. One or two meaningful queries per database type.

POSTMAN API Tool

POST: Create Researcher - MongoDB



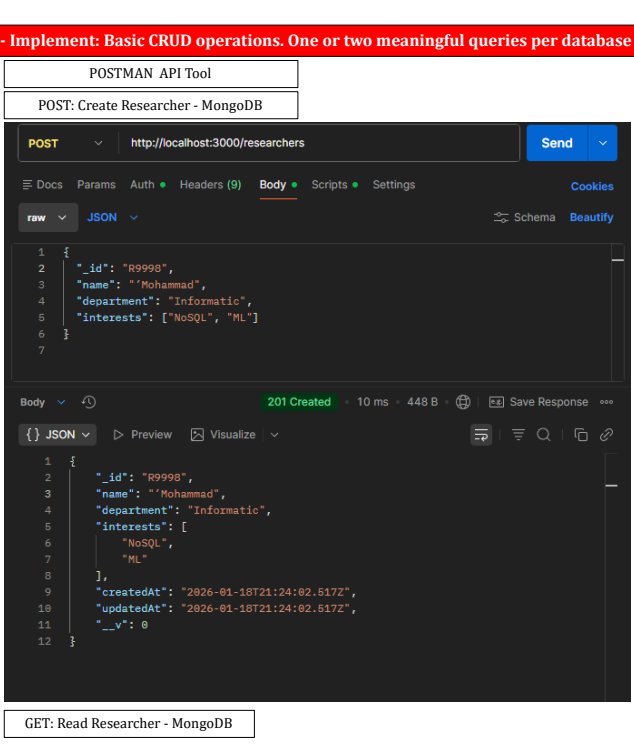
POST http://localhost:3000/researchers

```
{
  "_id": "R9998",
  "name": "'Mohammad",
  "department": "Informatic",
  "interests": ["NoSQL", "ML"]
}
```

201 Created 10 ms 448 B

```
{
  "_id": "R9998",
  "name": "'Mohammad",
  "department": "Informatic",
  "interests": [
    "NoSQL",
    "ML"
  ],
  "createdAt": "2026-01-18T21:24:02.517Z",
  "updatedAt": "2026-01-18T21:24:02.517Z",
  "__v": 0
}
```

GET: Read Researcher - MongoDB



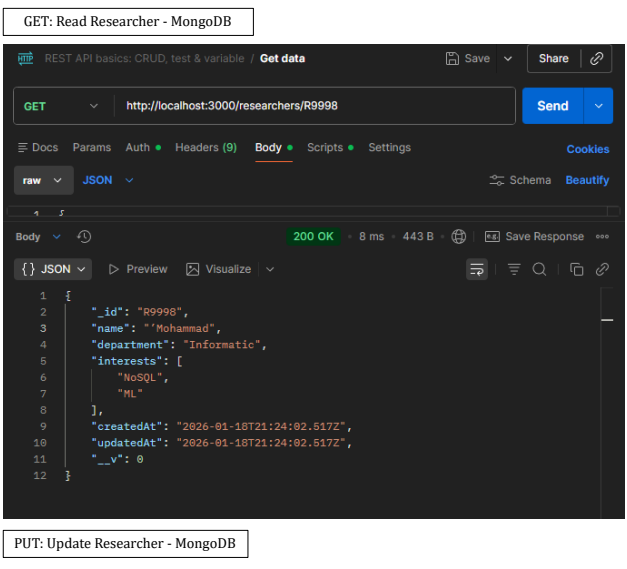
REST API basics: CRUD, test & variable / Get data

GET http://localhost:3000/researchers/R9998

200 OK 8 ms 443 B

```
{
  "_id": "R9998",
  "name": "'Mohammad",
  "department": "Informatic",
  "interests": [
    "NoSQL",
    "ML"
  ],
  "createdAt": "2026-01-18T21:24:02.517Z",
  "updatedAt": "2026-01-18T21:24:02.517Z",
  "__v": 0
}
```

PUT: Update Researcher - MongoDB



REST API basics: CRUD, test & variable / Get data

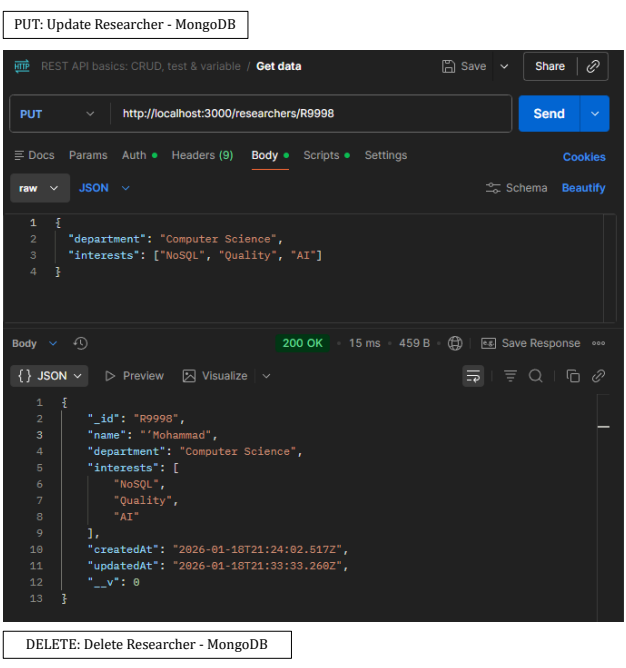
PUT http://localhost:3000/researchers/R9998

```
{
  "department": "Computer Science",
  "interests": ["NoSQL", "Quality", "AI"]
}
```

200 OK 15 ms 459 B

```
{
  "_id": "R9998",
  "name": "'Mohammad",
  "department": "Computer Science",
  "interests": [
    "NoSQL",
    "Quality",
    "AI"
  ],
  "createdAt": "2026-01-18T21:24:02.517Z",
  "updatedAt": "2026-01-18T21:33:33.260Z",
  "__v": 0
}
```

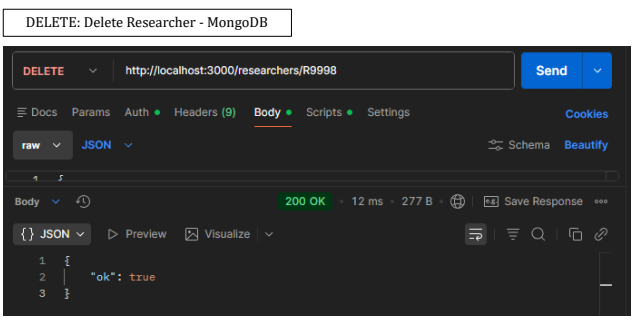
DELETE: Delete Researcher - MongoDB



DELETE http://localhost:3000/researchers/R9998

200 OK 12 ms 277 B

```
{
  "ok": true
}
```

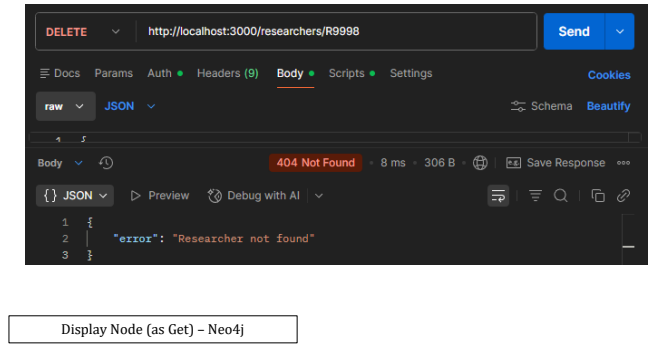


DELETE http://localhost:3000/researchers/R9998

404 Not Found 8 ms 308 B

```
{
  "error": "Researcher not found"
}
```

Display Node (as Get) – Neo4j



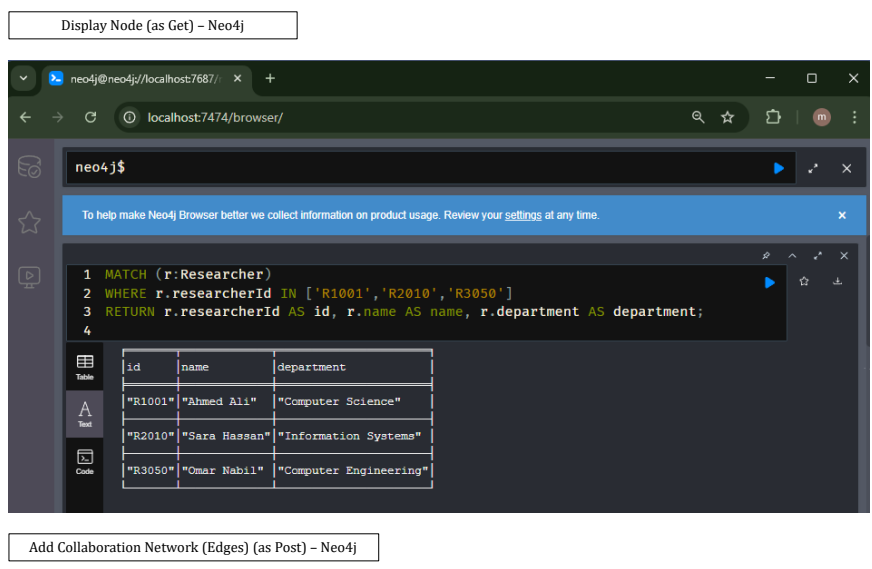
localhost:7474/browser/

To help make Neo4j Browser better we collect information on product usage. Review your settings at any time.

```
MATCH (r:Researcher)
WHERE r.researcherId IN ['R1001','R2010','R3050']
RETURN r.researcherId AS id, r.name AS name, r.department AS department;
```

```
id      | name          | department
"R1001" | "Ahmed Ali"   | "Computer Science"
"R2010" | "Sara Hassan" | "Information Systems"
"R3050" | "Omar Nabil"  | "Computer Engineering"
```

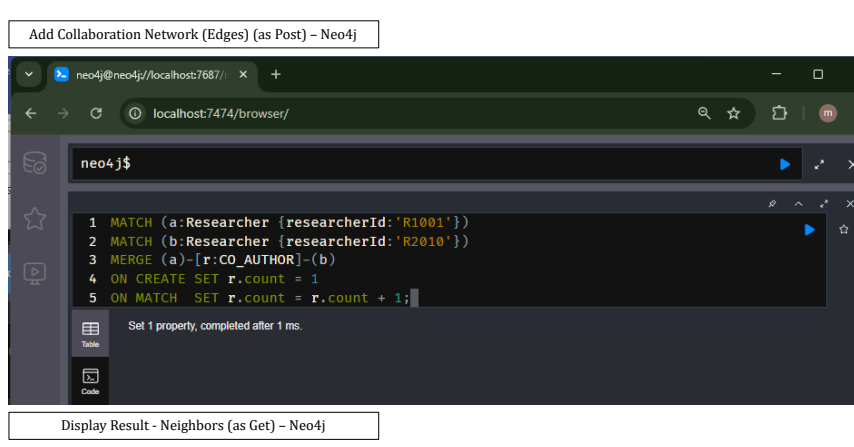
Add Collaboration Network (Edges) (as Post) – Neo4j



```
MATCH (a:Researcher {researcherId:'R1001'})
MATCH (b:Researcher {researcherId:'R2010'})
MERGE (a)-[r:CO_AUTHOR]-(b)
ON CREATE SET r.count = 1
ON MATCH SET r.count = r.count + 1;
```

Set 1 property, completed after 1 ms.

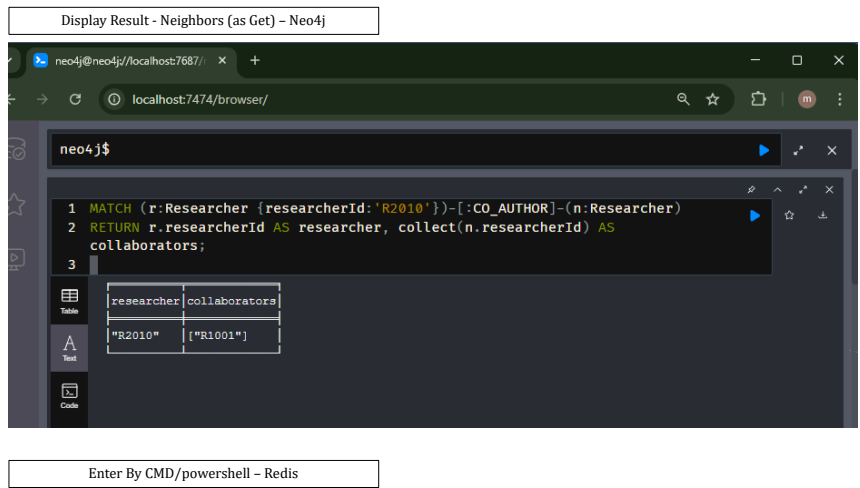
Display Result - Neighbors (as Get) – Neo4j



```
MATCH (r:Researcher {researcherId:'R2010'})-[:CO_AUTHOR]-(n:Researcher)
RETURN r.researcherId AS researcher, collect(n.researcherId) AS
collaborators;
```

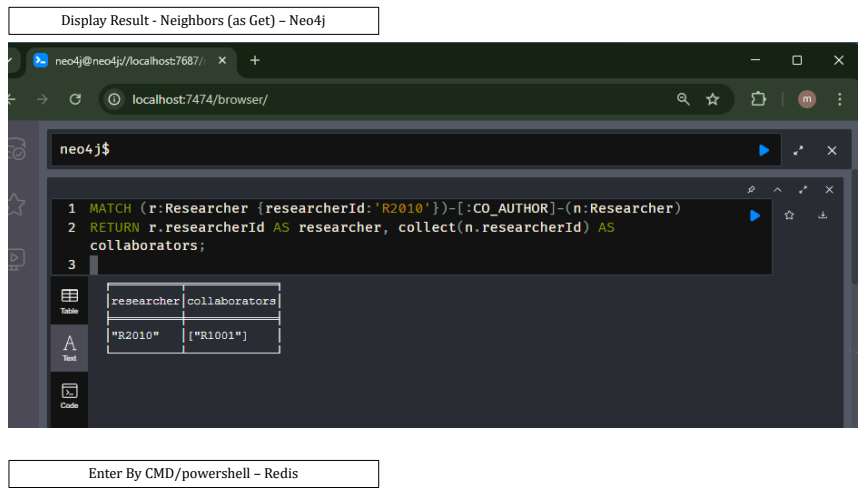
```
researcher | collaborators
"R2010"    | ["R1001"]
```

Enter By CMD/powershell – Redis



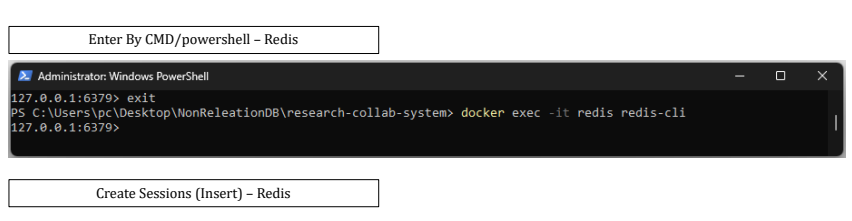
```
Administrator: Windows PowerShell
127.0.0.1:6379> exit
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> docker exec -it redis redis-cli
127.0.0.1:6379>
```

Create Sessions (Insert) – Redis



```
127.0.0.1:6379> SET session:SID001 R1001 EX 3600
OK
127.0.0.1:6379> SET session:SID002 R2010 EX 1800
OK
127.0.0.1:6379>
127.0.0.1:6379>
```

Display Sessions (Get) – Redis



```
127.0.0.1:6379> GET session:SID001
R1001
127.0.0.1:6379> TTL session:SID001
3388
127.0.0.1:6379>
```

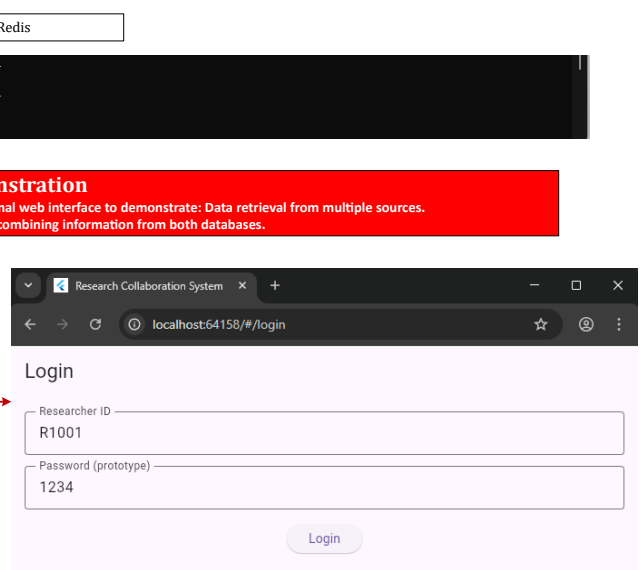
4.Integration and Demonstration

- Build a simple command-line or minimal web interface to demonstrate: Data retrieval from multiple sources.
- Example: show a researcher's profile combining information from both databases.

Login Page:
By user ID & Pass fetched from MongoDB,
Create session & Generate Token in RedisDB

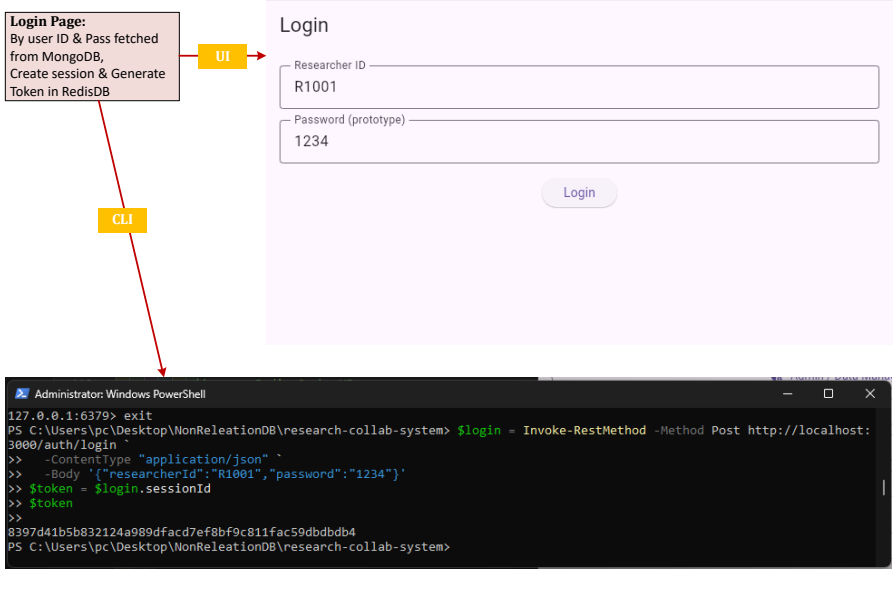
UI

CLI



Research Collaboration System
localhost:64158/#/login

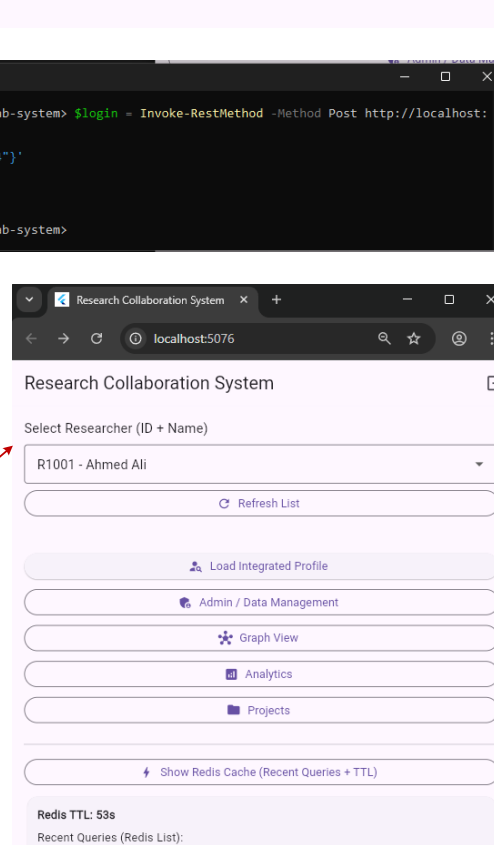
Login
Researcher ID: R1001
Password (prototype): 1234
Login



```
Administrator: Windows PowerShell
127.0.0.1:6379> exit
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> $login = Invoke-RestMethod -Method Post http://localhost:
3000/auth/login `
>>   -ContentType "application/json" `
>>   -Body '{"researcherId":"R1001","password":"1234"}'
>> $token = $login.sessionId
>> $token
>>
8397d41b5b832124a989dfacd7ef8bf9c811fac59dbdbdb4
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system>
```

Home Page:
- Select from Dropdown List A user for load integrated profile,
- Button for Admin Page
- Visual Graph
- Analytics
- Projects
- Show Redis Cache
- Logout

UI



Research Collaboration System
localhost:5076

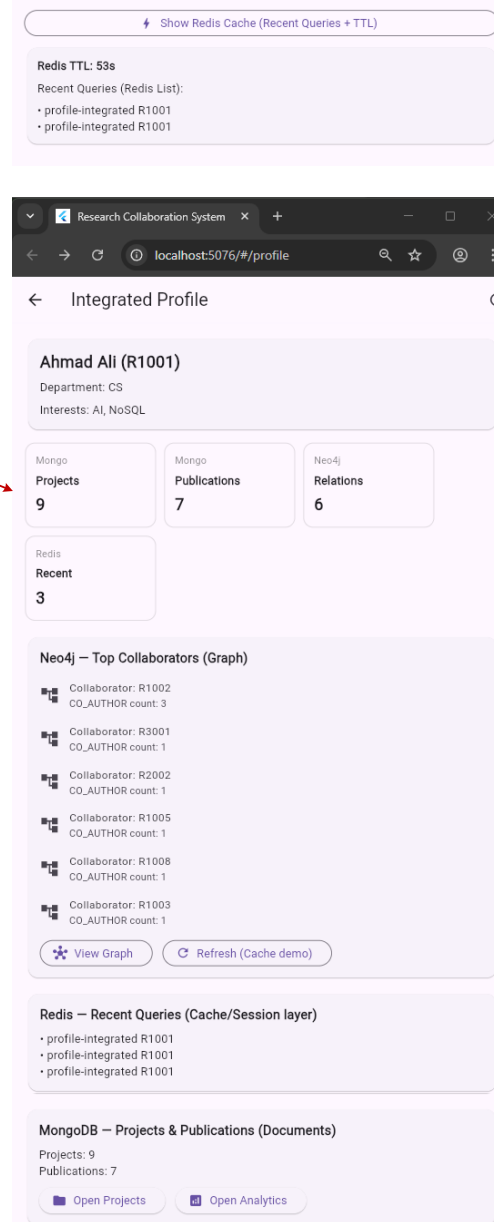
Research Collaboration System
Select Researcher (ID + Name)
R1001 - Ahmed Ali
Refresh List
Load Integrated Profile
Admin / Data Management
Graph View
Analytics
Projects
Show Redis Cache (Recent Queries + TTL)

Redis TTL: 53s
Recent Queries (Redis List):
- profile-integrated R1001
- profile-integrated R1001

Profile Page:
- info [dept, Interest] //Mongo
- Statistics //Mongo& Neo4j
- Relations& Count & Visual Graph //Neo4j
- Recent Queries //Redis
- Projects &Publication for selected user //Mongo

UI

CLI



localhost:5076/#/profile

Integrated Profile

Ahmad Ali (R1001)
Department: CS
Interests: AI, NoSQL

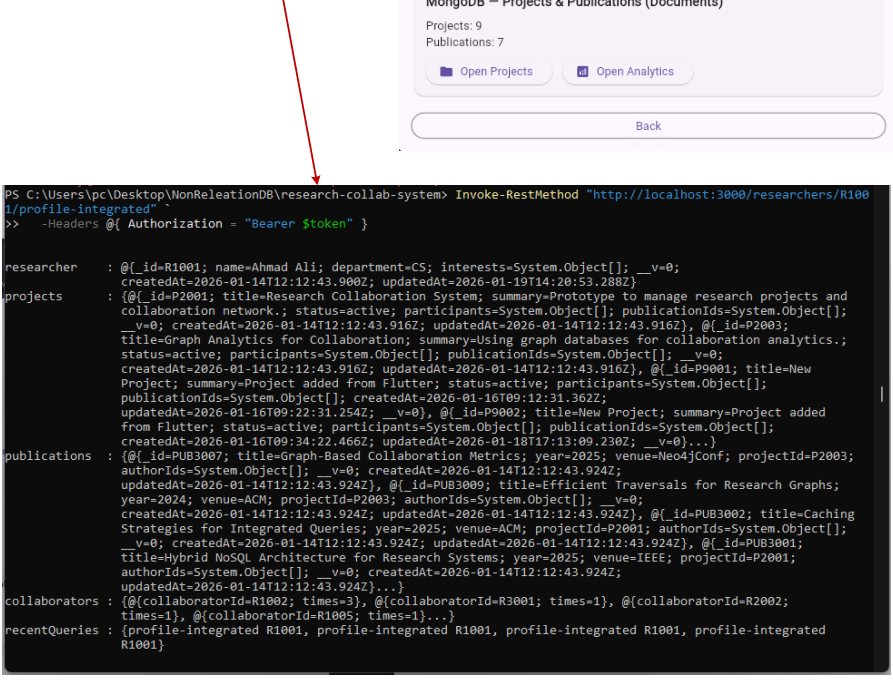
Mongo	Mongo	Neo4j
Projects	Publications	Relations
9	7	6

Redis
Recent
3

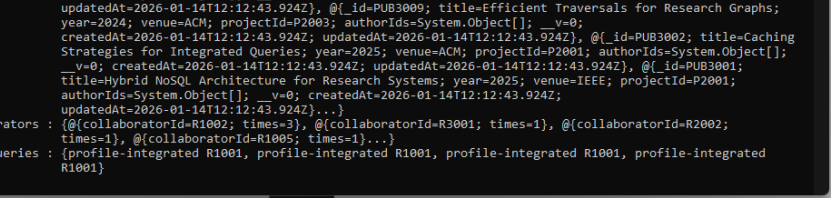
Neo4j — Top Collaborators (Graph)
Collaborator: R1002 — CO_AUTHOR count: 3
Collaborator: R3001 — CO_AUTHOR count: 1
Collaborator: R2002 — CO_AUTHOR count: 1
Collaborator: R1005 — CO_AUTHOR count: 1
Collaborator: R1008 — CO_AUTHOR count: 1
Collaborator: R1003 — CO_AUTHOR count: 1
View Graph
Refresh (Cache demo)

Redis — Recent Queries (Cache/Session layer)
- profile-integrated R1001
- profile-integrated R1001
- profile-integrated R1001

MongoDB — Projects & Publications (Documents)
Projects: 9
Publications: 7
Open Projects
Open Analytics
Back



```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/researchers/R100
1/profile-integrated" `
>>   -Headers @{ Authorization = "Bearer $token" }

researcher    : @{_id=R1001; name=Ahmad Ali; department=CS; interests=System.Object[]; __v=0;
                createdAt=2026-01-14T12:12:43.900Z; updatedAt=2026-01-19T14:20:53.288Z}
projects      : {@{_id=P2001; title=Research Collaboration System; summary=Prototype to manage research projects and
                collaboration network.; status=active; participants=System.Object[]; publicationIds=System.Object[];
                __v=0; createdAt=2026-01-14T12:12:43.916Z; updatedAt=2026-01-14T12:12:43.916Z}, @{_id=P2003;
                title=Graph Analytics for Collaboration; summary=Using graph databases for collaboration analytics.;
                status=active; participants=System.Object[]; publicationIds=System.Object[]; __v=0;
                createdAt=2026-01-14T12:12:43.916Z; updatedAt=2026-01-14T12:12:43.916Z}, @{_id=P9001; title=New
                Project; summary=Project added from Flutter; status=active; participants=System.Object[];
                publicationIds=System.Object[]; createdAt=2026-01-16T09:12:31.362Z;
                updatedAt=2026-01-16T09:22:31.254Z; __v=0}, @{_id=P9002; title=New Project; summary=Project added
                from Flutter; status=active; participants=System.Object[]; publicationIds=System.Object[];
                createdAt=2026-01-16T09:34:22.466Z; updatedAt=2026-01-18T17:13:09.230Z; __v=0}...}
publications  : {@{_id=PUB3007; title=Graph-Based Collaboration Metrics; year=2025; venue=Neo4jConf; projectId=P2003;
                authorIds=System.Object[]; __v=0; createdAt=2026-01-14T12:12:43.924Z;
                updatedAt=2026-01-14T12:12:43.924Z}, @{_id=PUB3009; title=Efficient Traversals for Research Graphs;
                year=2024; venue=ACM; projectId=P2003; authorIds=System.Object[]; __v=0;
                createdAt=2026-01-14T12:12:43.924Z; updatedAt=2026-01-14T12:12:43.924Z}, @{_id=PUB3002; title=Caching
                Strategies for Integrated Queries; year=2025; venue=ACM; projectId=P2001; authorIds=System.Object[];
                __v=0; createdAt=2026-01-14T12:12:43.924Z; updatedAt=2026-01-14T12:12:43.924Z}, @{_id=PUB3001;
                title=Hybrid NoSQL Architecture for Research Systems; year=2025; venue=IEEE; projectId=P2001;
                authorIds=System.Object[]; __v=0; createdAt=2026-01-14T12:12:43.924Z;
                updatedAt=2026-01-14T12:12:43.924Z}...}
collaborators : {@{collaboratorId=R1002; times=3}, @{collaboratorId=R3001; times=1}, @{collaboratorId=R2002;
                times=1}, @{collaboratorId=R1005; times=1}...}
recentQueries : {profile-integrated R1001, profile-integrated R1001, profile-integrated R1001, profile-integrated
                R1001}
```

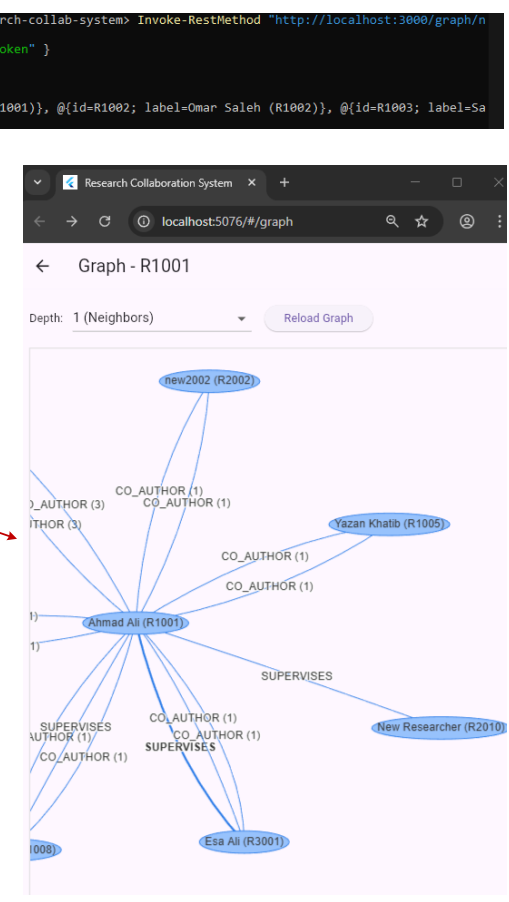


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/graph/n
eighborhood/R1001?depth=1" `
>>   -Headers @{ Authorization = "Bearer $token" }

id    depth nodes
--    ----- -----
R1001     1 {@{id=R1001; label=Ahmad Ali (R1001)}, @{id=R1002; label=Omar Saleh (R1002)}, @{id=R1003; label=Sa
ra Na...
```

Graph Page(Neo4j):
- Node of ID
- Neighborhoods' id
- Relationships (Co_Auther OR Supervisors)
- Depth of relation //1 or 2

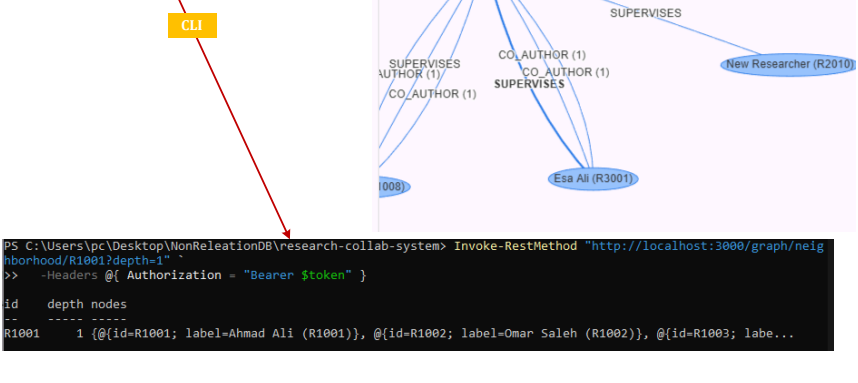
UI

CLI



localhost:5076/#/graph

Graph - R1001
Depth: 1 (Neighbors)
Reload Graph

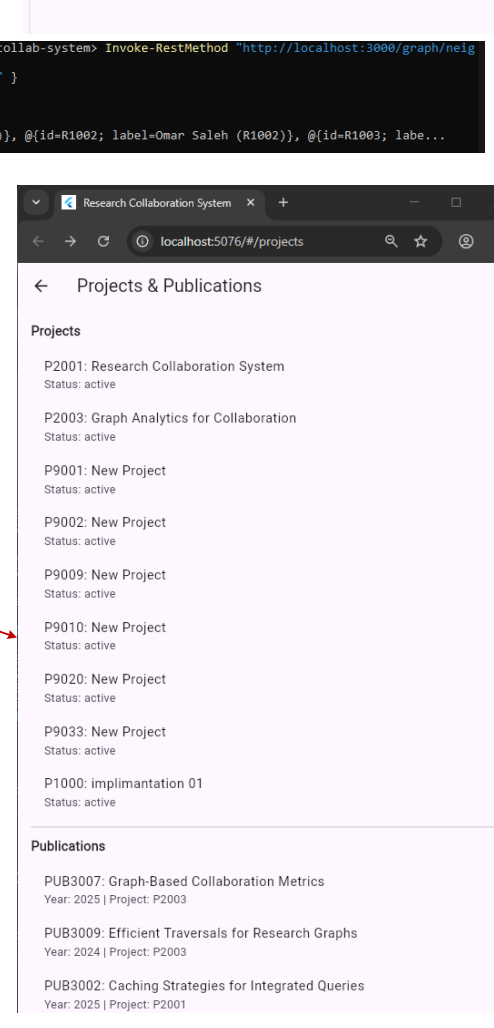


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/graph/neig
hborhood/R1001?depth=1" `
>>   -Headers @{ Authorization = "Bearer $token" }

id    depth nodes
--    ----- -----
R1001     1 {@{id=R1001; label=Ahmad Ali (R1001)}, @{id=R1002; label=Omar Saleh (R1002)}, @{id=R1003; labe...
```

Projects & Publications Page:
- List of projects related & Status
- List of publications related & Status

UI

CLI

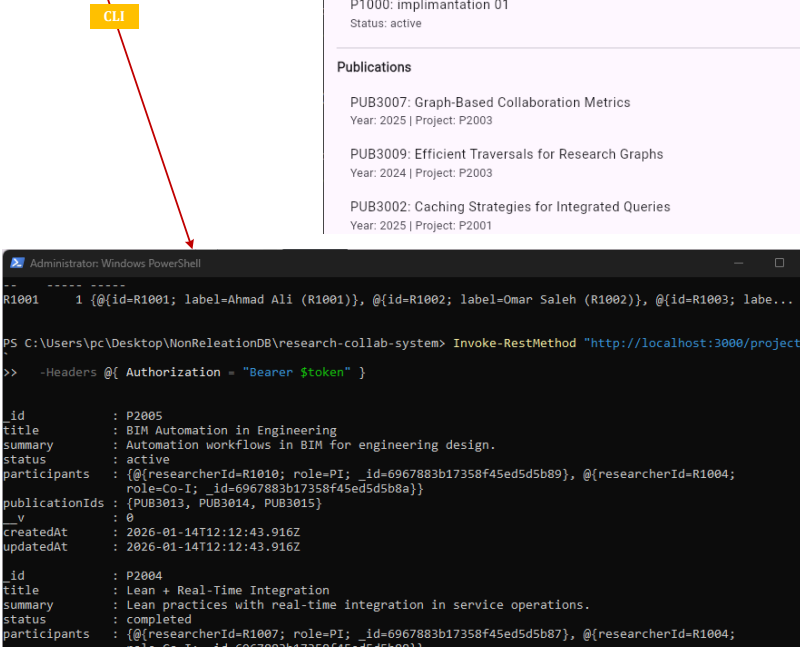


localhost:5076/#/projects

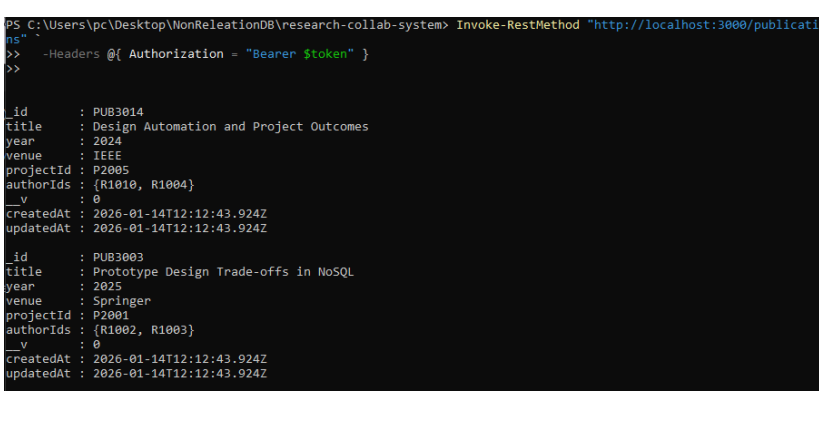
Projects & Publications

Projects
P2001: Research Collaboration System — Status: active
P2003: Graph Analytics for Collaboration — Status: active
P9001: New Project — Status: active
P9002: New Project — Status: active
P9009: New Project — Status: active
P9010: New Project — Status: active
P9020: New Project — Status: active
P9033: New Project — Status: active
P1000: implimantation 01 — Status: active

Publications
PUB3007: Graph-Based Collaboration Metrics — Year: 2025 | Project: P2003
PUB3009: Efficient Traversals for Research Graphs — Year: 2024 | Project: P2003
PUB3002: Caching Strategies for Integrated Queries — Year: 2025 | Project: P2001



```
Administrator: Windows PowerShell
R1001     1 {@{id=R1001; label=Ahmad Ali (R1001)}, @{id=R1002; label=Omar Saleh (R1002)}, @{id=R1003; labe...

PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/projects"
>>   -Headers @{ Authorization = "Bearer $token" }

_id            : P2005
title          : BIM Automation in Engineering
summary        : Automation workflows in BIM for engineering design.
status         : active
participants   : {@{researcherId=R1010; role=PI; _id=6967883b17358f45ed5db89}, @{researcherId=R1004;
                 role=Co-I; _id=6967883b17358f45ed5db8a}}
publicationIds : {PUB3013, PUB3014, PUB3015}
__v            : 0
createdAt      : 2026-01-14T12:12:43.916Z
updatedAt      : 2026-01-14T12:12:43.916Z

_id            : P2004
title          : Lean + Real-Time Integration
summary        : Lean practices with real-time integration in service operations.
status         : completed
participants   : {@{researcherId=R1007; role=PI; _id=6967883b17358f45ed5db87}, @{researcherId=R1004;
```

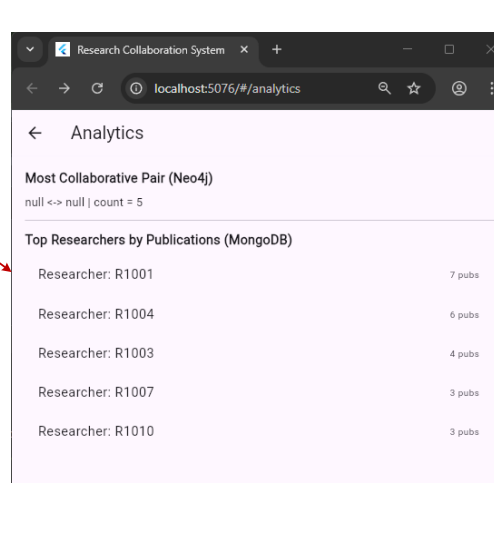


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/publicatio
ns" `
>>   -Headers @{ Authorization = "Bearer $token" }
>>

_id       : PUB3014
title     : Design Automation and Project Outcomes
year      : 2024
venue     : IEEE
projectId : P2005
authorIds : {R1010, R1004}
__v       : 0
createdAt : 2026-01-14T12:12:43.924Z
updatedAt : 2026-01-14T12:12:43.924Z

_id       : PUB3003
title     : Prototype Design Trade-offs in NoSQL
year      : 2025
venue     : Springer
projectId : P2001
authorIds : {R1002, R1003}
__v       : 0
createdAt : 2026-01-14T12:12:43.924Z
updatedAt : 2026-01-14T12:12:43.924Z
```

Analytics Page (Neo4j):
- Top Collaborative Pair
- Top researchers by publications with Counter

UI

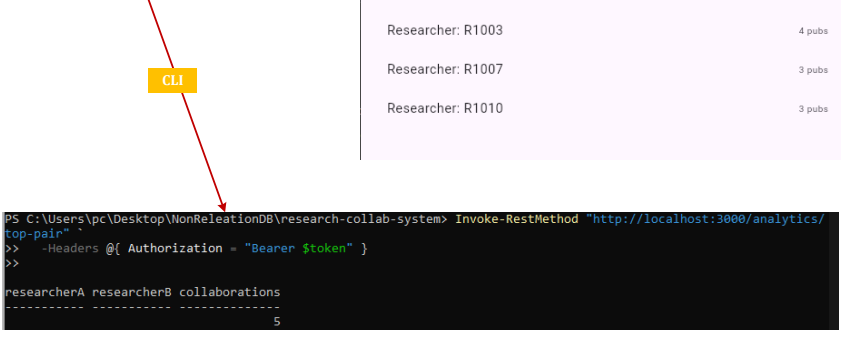
CLI



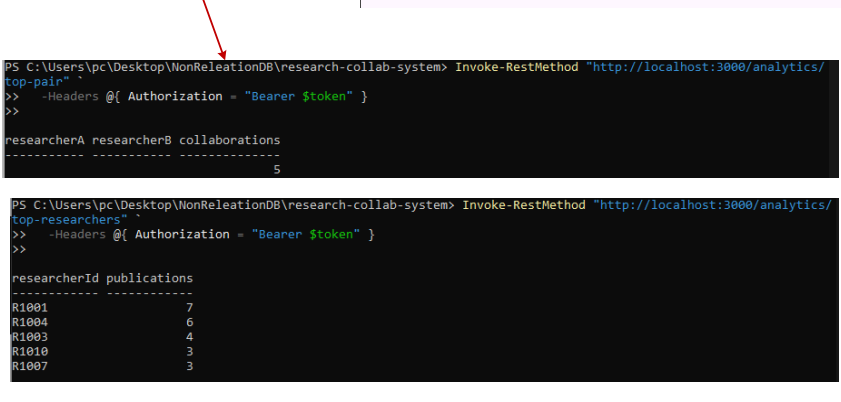
localhost:5076/#/analytics

Analytics
Most Collaborative Pair (Neo4j)
null <-> null | count = 5
Top Researchers by Publications (MongoDB)

Researcher: R1001	7 pubs
Researcher: R1004	6 pubs
Researcher: R1003	4 pubs
Researcher: R1007	3 pubs
Researcher: R1010	3 pubs



```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/analytics/
top-pair" `
>>   -Headers @{ Authorization = "Bearer $token" }
>>

researcherA researcherB collaborations
----------- ----------- --------------
                                     5
```

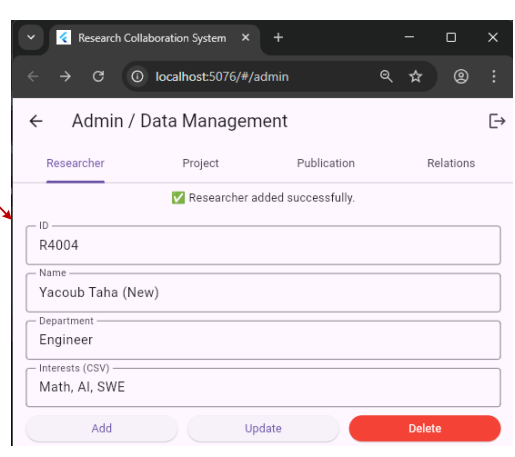


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod "http://localhost:3000/analytics/
top-researchers" `
>>   -Headers @{ Authorization = "Bearer $token" }
>>

researcherId publications
------------ ------------
R1001                   7
R1004                   6
R1003                   4
R1010                   3
R1007                   3
```

Admin Page (Mongo&Neo4j):
- 4 Taps
- Researcher(Add, Update, Delete)
- Project (Add, Update, Delete)
- Publication(Add, Update, Delete)
- Relation(Add, Update, Delete)

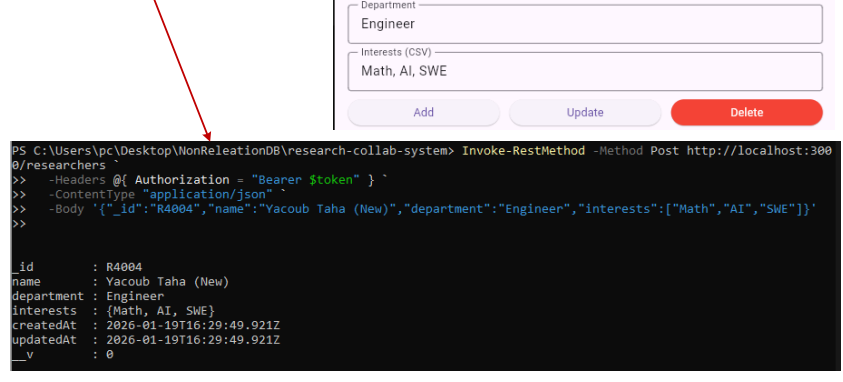
UI

CLI



localhost:5076/#/admin

Admin / Data Management
Researcher | Project | Publication | Relations
✅ Researcher added successfully.
ID: R4004
Name: Yacoub Taha (New)
Department: Engineer
Interests (CSV): Math, AI, SWE
Add | Update | Delete

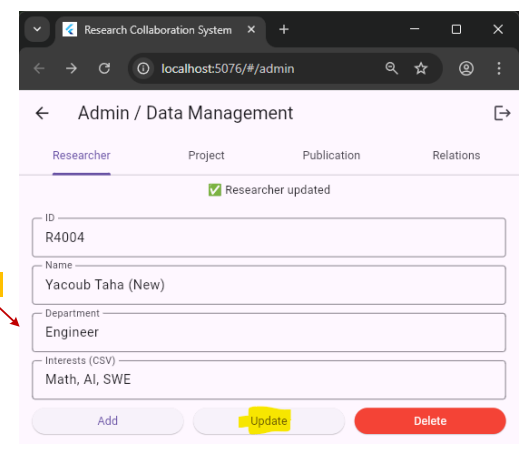


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod -Method Post http://localhost:300
0/researchers `
>>   -Headers @{ Authorization = "Bearer $token" } `
>>   -ContentType "application/json" `
>>   -Body '{"_id":"R4004","name":"Yacoub Taha (New)","department":"Engineer","interests":["Math","AI","SWE"]}'
>>

_id        : R4004
name       : Yacoub Taha (New)
department : Engineer
interests  : {Math, AI, SWE}
createdAt  : 2026-01-19T16:29:49.921Z
updatedAt  : 2026-01-19T16:29:49.921Z
__v        : 0
```

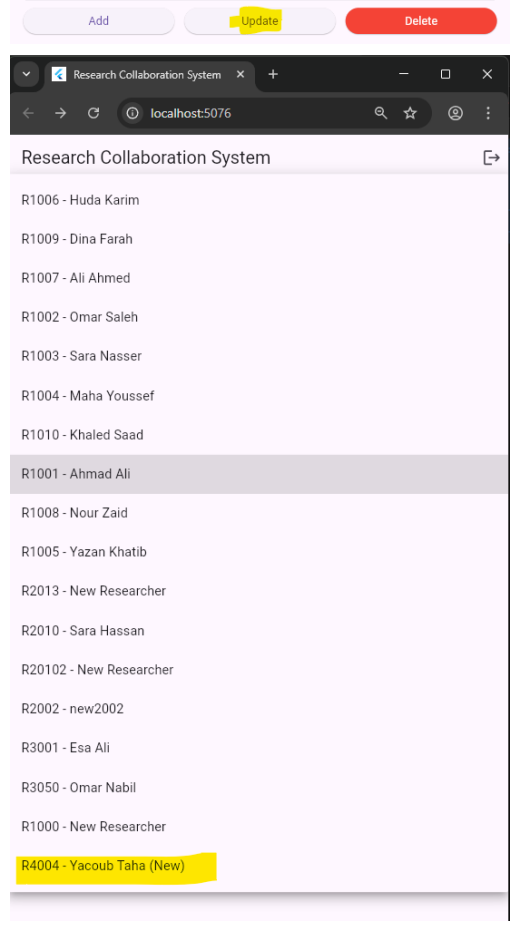
Admin Page (Mongo&Neo4j):
- Update

UI

CLI

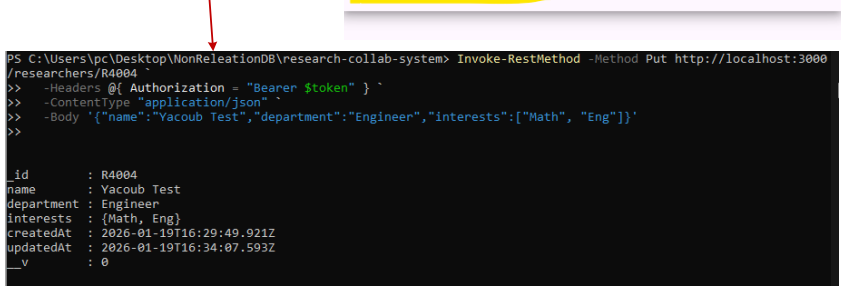


Admin / Data Management
Researcher | Project | Publication | Relations
✅ Researcher updated
ID: R4004
Name: Yacoub Taha (New)
Department: Engineer
Interests (CSV): Math, AI, SWE
Add | Update | Delete



localhost:5076

Research Collaboration System
R1006 - Huda Karim
R1009 - Dina Farah
R1007 - Ali Ahmed
R1002 - Omar Saleh
R1003 - Sara Nasser
R1004 - Maha Youssef
R1010 - Khaled Saad
R1001 - Ahmad Ali
R1008 - Nour Zaid
R1005 - Yazan Khatib
R2013 - New Researcher
R2010 - Sara Hassan
R20102 - New Researcher
R2002 - new2002
R3001 - Esa Ali
R3050 - Omar Nabil
R1000 - New Researcher
R4004 - Yacoub Taha (New)

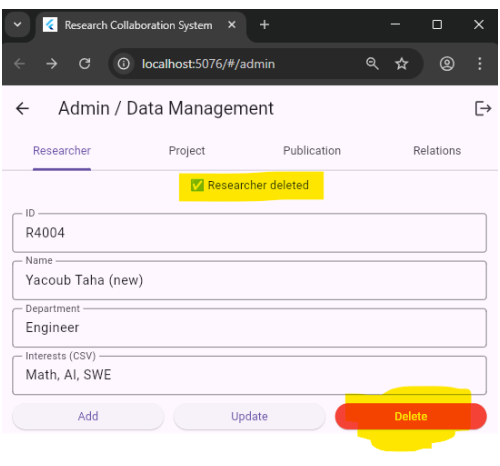


```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod -Method Put http://localhost:3000
/researchers/R4004 `
>>   -Headers @{ Authorization = "Bearer $token" } `
>>   -ContentType "application/json" `
>>   -Body '{"name":"Yacoub Test","department":"Engineer","interests":["Math", "Eng"]}'
>>

_id        : R4004
name       : Yacoub Test
department : Engineer
interests  : {Math, Eng}
createdAt  : 2026-01-19T16:29:49.921Z
updatedAt  : 2026-01-19T16:34:07.593Z
__v        : 0
```

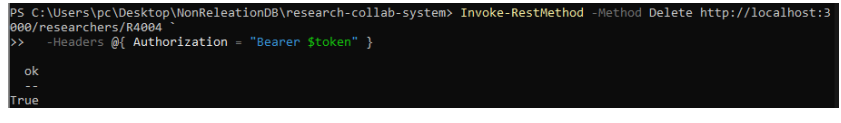
Admin Page (Mongo&Neo4j):
-Delete

UI

CLI



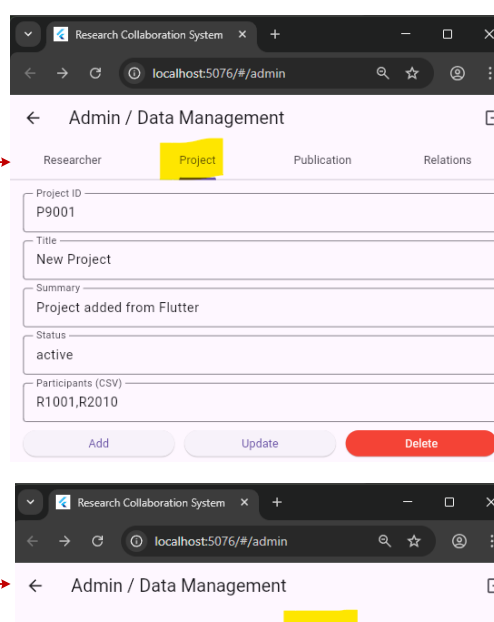
Admin / Data Management
Researcher | Project | Publication | Relations
✅ Researcher deleted
ID: R4004
Name: Yacoub Taha (new)
Department: Engineer
Interests (CSV): Math, AI, SWE
Add | Update | Delete



```
PS C:\Users\pc\Desktop\NonReleationDB\research-collab-system> Invoke-RestMethod -Method Delete http://localhost:3
000/researchers/R4004 `
>>   -Headers @{ Authorization = "Bearer $token" }

ok
--
True
```

Admin Page (Mongo&Neo4j):
Add, Update, Delete

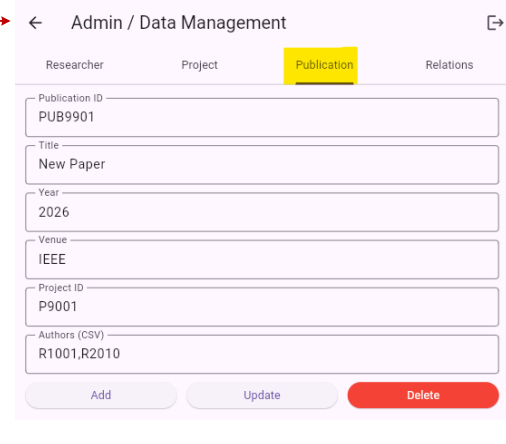
UI



Admin / Data Management
Researcher | Project | Publication | Relations
Project ID: P9001
Title: New Project
Summary: Project added from Flutter
Status: active
Participants (CSV): R1001,R2010
Add | Update | Delete

Admin Page (Mongo&Neo4j):
Add, Update, Delete

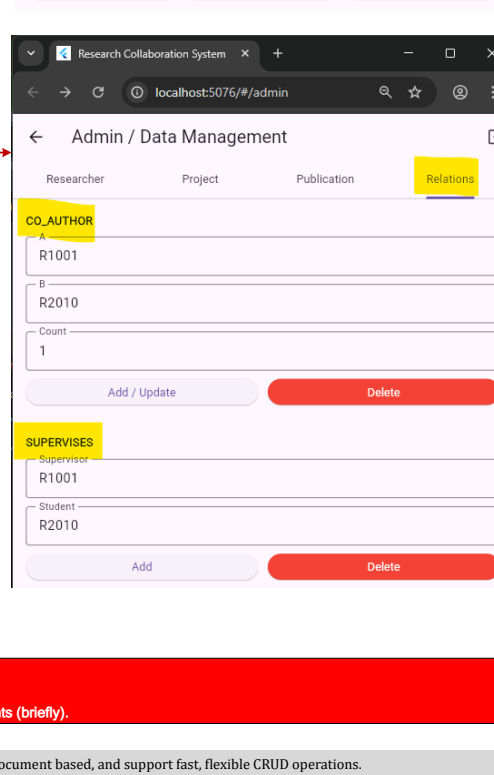
UI



Admin / Data Management
Researcher | Project | Publication | Relations
Publication ID: PUB9901
Title: New Paper
Year: 2026
Venue: IEEE
Project ID: P9001
Authors (CSV): R1001,R2010
Add | Update | Delete

Admin Page (Mongo&Neo4j):
Add, Update, Delete

UI



Admin / Data Management
Researcher | Project | Publication | Relations
CO_AUTHOR
A: R1001
B: R2010
Count: 1
Add / Update | Delete
SUPERVISES
Supervisor: R1001
Student: R2010
Add | Delete

5.Discussion

- Explain design trade-offs.
- Mention potential scalability or optimization improvements (briefly).

- I'm using MongoDB for the core entities because it document based, and support fast, flexible CRUD operations.
- and using Neo4j because relationship based queries and network analytics are hard to efficiently in MongoDB, while graph traversal is much faster.
- also using Redis for session management with TTL for session expiration and cashing profile to reduce load on MongoDB & Neo4j.
- The Main trade-off is increased system complexity and consistency when using multiple Data Base, but the goal of project to integrate and justify design.

- For scalability, we can add indexes in MongoDB on frequently field.
- For Neo4j also add indexes on Nodes/Researchers_ID to improve traversal performance.
- Regarding Redis to cashe the integrated profile results, so repeated request return faster without query by MongoDB & Neo4j every time, when data changes we clear the cashe to avoid outdated results.
- Supporting Horizontally scaling for each type of Data Base by clustering and Sharding technique.

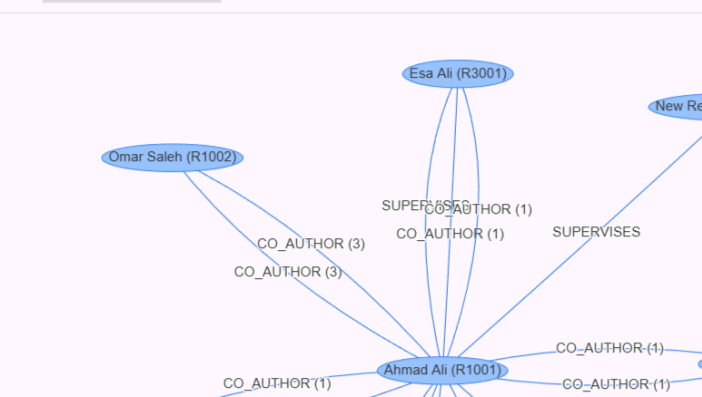
Bonus is possible if:

- You implement a small cluster or multi-node setup.
- You show creative or original design ideas.

DOCKER (compose)
MongoDB Replica Set

[mongo1] Primary
[mongo2] Secondary
[mongo3] Secondary

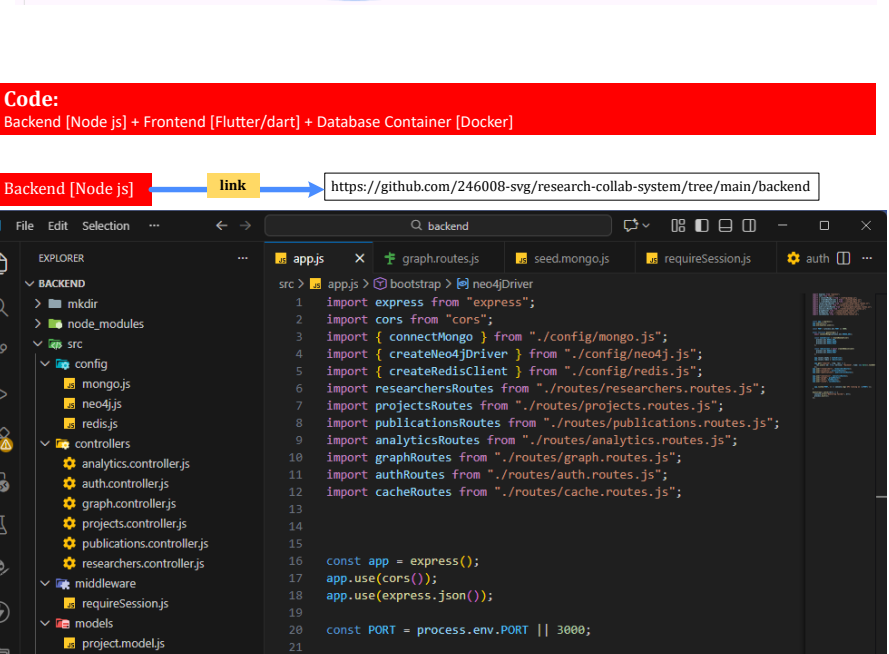
MongoDB configured as Replica Set with 3 nodes, so the system supports replication and basic fault tolerance.



```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
...   health: m.health
... }))
...
[
  { name: 'mongo1:27017', stateStr: 'SECONDARY', health: 1 },
  { name: 'mongo2:27017', stateStr: 'SECONDARY', health: 1 },
  { name: 'mongo3:27017', stateStr: 'PRIMARY', health: 1 }
]
rs0 [direct: secondary] test>
```

- You show creative or original design ideas.

I visualize the collaboration network stored in Neo4j as an interactive graph. This helps understand research relationships and identify highly connected researchers by library in UI flutter as demo show with ability to change depth of relations.



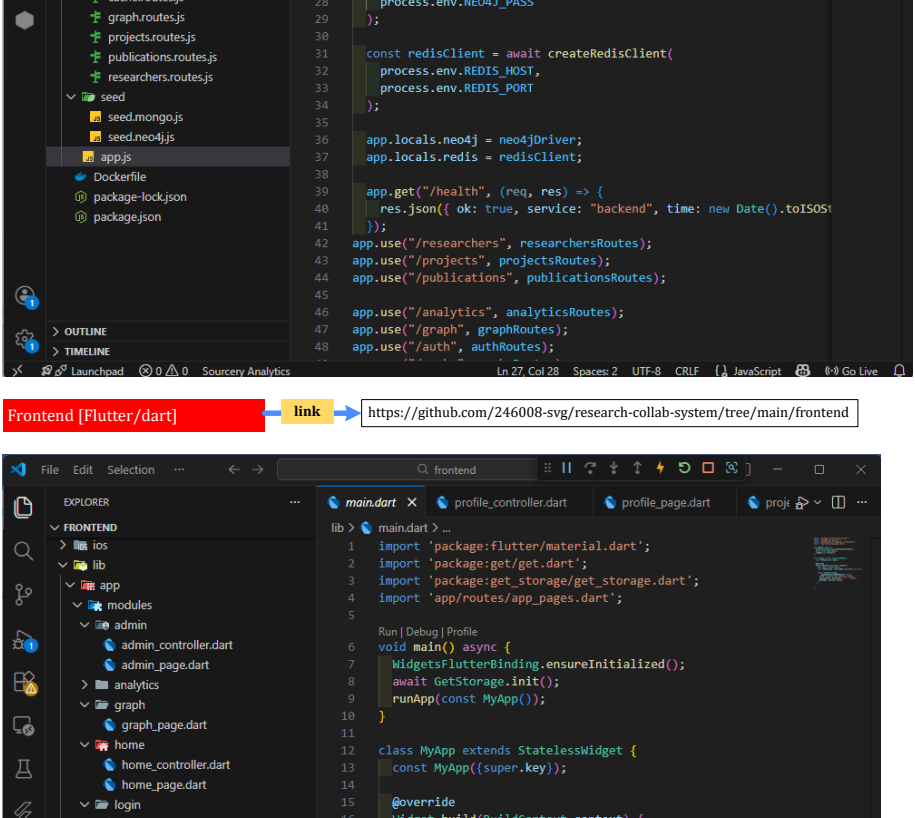
localhost:39293/#/graph

Graph - R1001
Depth: 1 (Neighbors)
Reload Graph

Code:

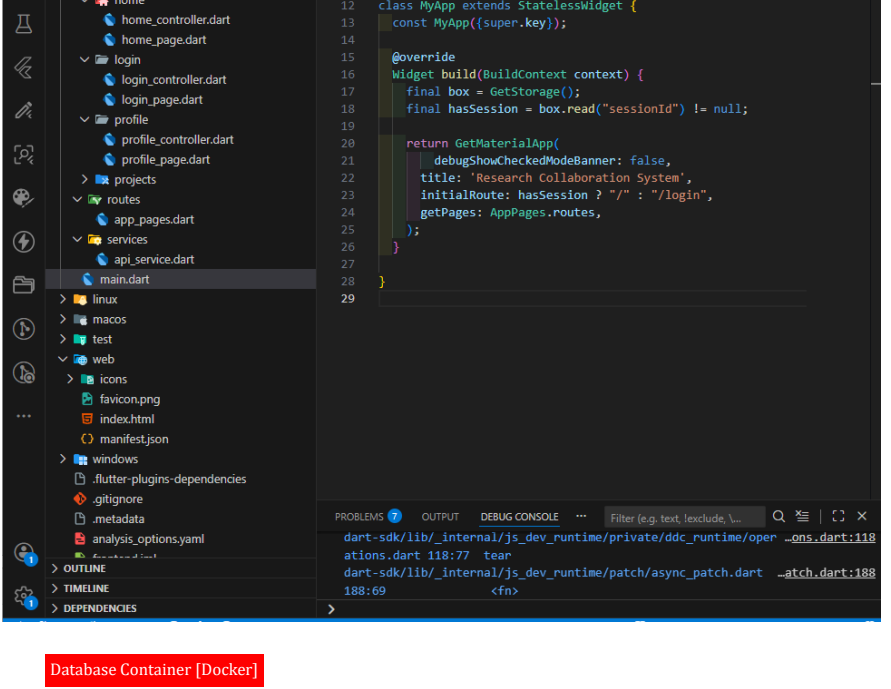
Backend [Node js] + Frontend [Flutter/dart] + Database Container [Docker]

Backend [Node js] link https://github.com/246008-svg/research-collab-system/tree/main/backend



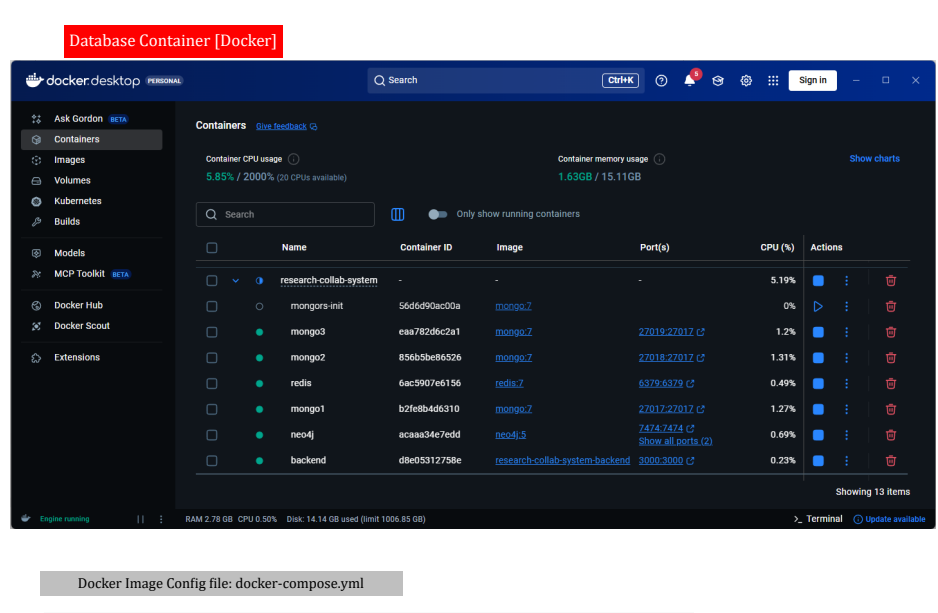
```
import express from "express";
import cors from "cors";
import { connectMongo } from "./config/mongo.js";
import { createNeo4jDriver } from "./config/neo4j.js";
import { createRedisClient } from "./config/redis.js";
import researchersRoutes from "./routes/researchers.routes.js";
import projectsRoutes from "./routes/projects.routes.js";
import publicationsRoutes from "./routes/publications.routes.js";
import analyticsRoutes from "./routes/analytics.routes.js";
import graphRoutes from "./routes/graph.routes.js";
import authRoutes from "./routes/auth.routes.js";
import cacheRoutes from "./routes/cache.routes.js";



const app = express();
app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 3000;

async function bootstrap() {
  await connectMongo(process.env.MONGO_URI);

  const neo4jDriver = createNeo4jDriver(
    process.env.NEO4J_URI,
    process.env.NEO4J_USER,
    process.env.NEO4J_PASS
  );

  const redisClient = await createRedisClient(
    process.env.REDIS_HOST,
    process.env.REDIS_PORT
  );

  app.locals.neo4j = neo4jDriver;
  app.locals.redis = redisClient;

  app.get("/health", (req, res) => {
    res.json({ ok: true, service: "backend", time: new Date().toISOS
  });

  app.use("/researchers", researchersRoutes);
  app.use("/projects", projectsRoutes);
  app.use("/publications", publicationsRoutes);

  app.use("/analytics", analyticsRoutes);
  app.use("/graph", graphRoutes);
  app.use("/auth", authRoutes);
```

Frontend [Flutter/dart] link https://github.com/246008-svg/research-collab-system/tree/main/frontend



```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:get_storage/get_storage.dart';
import 'app/routes/app_pages.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await GetStorage.init();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final box = GetStorage();
    final hasSession = box.read("sessionId") != null;

    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Research Collaboration System',
      initialRoute: hasSession ? "/" : "/login",
      getPages: AppPages.routes,
    );
  }
}
```

Database Container [Docker]



Docker Image Config file: docker-compose.yml

```
services:
  mongo1:
    image: mongo:7
    container_name: mongo1
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports: ["27017:27017"]
    volumes: ["mongo1_data:/data/db"]
    networks: [appnet]

  mongo2:
    image: mongo:7
    container_name: mongo2
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports: ["27018:27017"]
    volumes: ["mongo2_data:/data/db"]
    networks: [appnet]

  mongo3:
    image: mongo:7
    container_name: mongo3
    command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]
    ports: ["27019:27017"]
    volumes: ["mongo3_data:/data/db"]
    networks: [appnet]

  mongo-init:
    image: mongo:7
    container_name: mongo-init
    depends_on: [mongo1, mongo2, mongo3]
    networks: [appnet]
    entrypoint: >
      bash -lc "
      sleep 5 &&
      mongosh --host mongo1:27017 --eval '
        rs.initiate({
          _id: \"rs0\",
          members: [
            { _id: 0, host: \"mongo1:27017\" },
            { _id: 1, host: \"mongo2:27017\" },
            { _id: 2, host: \"mongo3:27017\" }
          ]
        })

  neo4j:
    image: neo4j:5
    container_name: neo4j
    ports:
      - "7474:7474"
      - "7687:7687"
    environment:
      - NEO4J_AUTH=neo4j/password123
    volumes:
      - neo4j_data:/data
    networks: [appnet]

  redis:
    image: redis:7
    container_name: redis
```